iPhone Duo来了，腾讯 Kuikly 助 App 从容适配

原创 腾讯程序员 腾讯技术工程 2026年9月9日 21:03 广西 102人



> Kuikly 是腾讯开源的高性能跨端框架，基于 Kotlin Multiplatform 技术，覆盖 Android、iOS、HarmonyOS、H5、微信小程序、Mac 六大平台，支撑业务日活用户超 5 亿。当折叠屏适配潮加速到来，Kuikly 如何帮助 App 从容适配——单次开发，即可覆盖 Android、iOS、HarmonyOS 多端，让同一套页面自然适应折叠、展开、分屏和小窗等不同形态。

前言

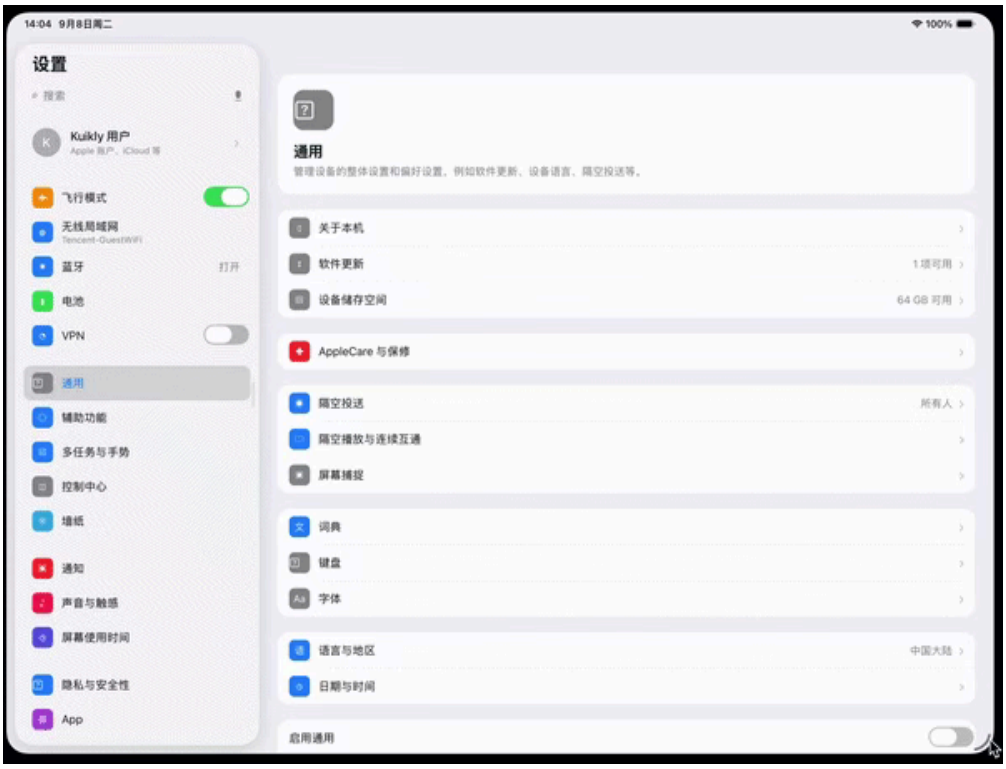
今天凌晨，在苹果秋季发布会上，苹果发布了全新的折叠设备形态。不同于常规的硬件参数升级，折叠屏改变了移动设备的可用空间：设备可以在便携的小屏与展开的大屏之间切换，App 界面也需要从固定布局转向能够连续响应窗口变化的多形态设计。

在苹果入局之前，华为、三星、荣耀、OPPO、vivo 和小米等厂商已经持续推出折叠屏产品，迭代节奏不断加快，形态也日趋丰富。苹果的加入进一步扩展了折叠屏的行业覆盖范围，也使内容浏览、移动办公、多任务协同等场景的大小屏适配成为更多 App 需要考虑的问题。

AI 也在改变移动终端的信息结构：从单一的问答入口，逐步发展为同时承载上下文资料、工具调用、生成结果和人工确认的复合任务界面。与此同时，远程办公、云桌面和跨设备协同也需要在有限空间内并行展示更多信息。随着这些场景持续发展，移动终端对显示空间的需求将进一步增加，折叠屏与大屏可能逐步成为 AI 超级终端和移动生产力设备的重要发展方向。

趋势背后，一个更加现实的问题也摆在了开发者面前：面对折叠屏带来的可变窗口场景，当手机可以在小屏与大屏之间切换，App 应该如何跟着"展开"？现有页面能否自然适应？布局、路由、动画和手势是否都要重新开发？Android、iOS、HarmonyOS 多平台又是否需要分别适配？

Kuikly 在设计之初便考虑了窗口变化与多形态适配，并已具备成熟的折叠屏支持能力。基于 Kotlin Multiplatform 与共享 UI 体系，开发者可以在共享 Kotlin 代码中统一处理尺寸响应、状态管理和多形态布局，将核心逻辑复用于多个平台之上。


14:04 9月8日周二
100%
设置
搜索
Kuikly 用户
Apple 账户、iCloud 等
飞行模式
无线局域网
Tencent-GuestWiFi
蓝牙
打开
电池
VPN
通用
辅助功能
多任务与手势
控制中心
墙纸
通知
声音与触感
屏幕使用时间
隐私与安全性
App
通用
管理设备的整体设置和偏好设置，例如软件更新、设备语言、隔空投送等。
关于本机
软件更新
1 项可用
设备储存空间
64 GB 可用
AppleCare 与保修
隔空投送
所有人
隔空播放与连续互通
屏幕捕捉
词典
键盘
字体
语言与地区
中国大陆
日期与时间
启用通用


2:17
设置
搜索
Kuikly 用户
Apple 账户、iCloud 等
飞行模式
无线局域网
Tencent-GuestWiFi
蓝牙
打开
电池
VPN
通用
辅助功能
多任务与手势
控制中心
通用
通用
管理设备的整体设置和偏好设置，例如软件更新、设备语言、隔空投送等。
关于本机
软件更新
1 项可用
设备储存空间
64 GB 可用
AppleCare 与保修
隔空投送
所有人
隔空播放与连续互通
屏幕捕捉



基于 Kuikly 实现的自适应屏幕设置页在 iOS / 安卓 / 鸿蒙上的表现

一、折叠屏适配潮将至，App 如何应对?

1.1 苹果跟进，折叠屏适配将成为多端共同需求

过去，折叠屏适配主要集中在部分 Android 与 HarmonyOS 设备。随着苹果跟进，折叠屏将进一步覆盖主流移动设备，App 也将迎来一轮集中适配潮。

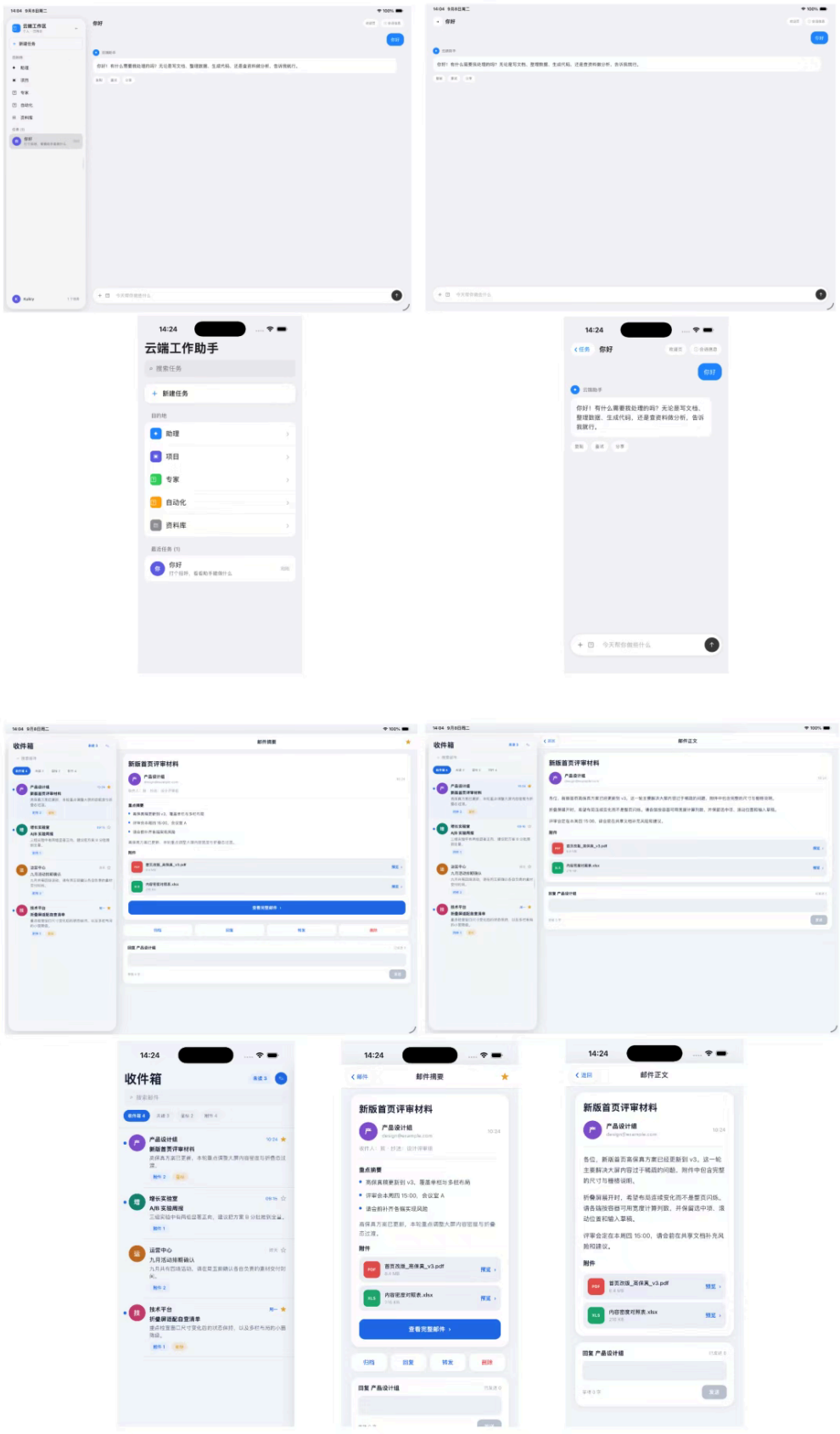
这次适配不只是支持一款新设备，而是要让现有页面同时应对折叠、展开、分屏等可变窗口形态，并在多平台上保持一致体验。

1.2 真正的痛点不只是布局

折叠屏适配看似是屏幕尺寸变化，实际会牵动整个页面体系：

- **改造工作量大：**首页、主 Tab、频道页等入口页面普遍需要重新组织导航、内容分区和信息密度，存量页面还要重新梳理布局断点与测试场景；
- **交互链路复杂：**单栏变多栏后，路由、状态恢复、动画、返回与手势都要同步调整；
- **多端成本放大：**同一套适配如果在各平台分别实现，还要覆盖多种窗口形态，开发和维护成本会快速增长。

因此，折叠屏适配既要保证形态切换时任务不中断，也要让新增空间真正提升信息展示与操作效率。如何以更低成本在多端实现这两个目标，正是 Kuikly 要解决的问题。



基于 Kuikly 实现的自适应 App 形态

二、折叠屏适配，Kuikly 能给你带来什么？

折叠屏适配并不是增加几条布局判断，而是一项同时跨越平台、窗口形态与交互方式的系统工程。对多端 App 而言，其复杂度更接近：

> 适配复杂度 = 平台数量 × 窗口形态 × 交互模式

平台构成第一层差异；折叠态、展开态、分屏、小窗构成第二层差异；触摸、返回手势等交互方式又构成第三层差异。任意一项业务调整，都可能需要在这些组合中重新实现和验证。

在传统方案中，平台、形态和交互方式每增加一项，都可能产生新的开发与验证组合。Kuikly 并非简单抹平这些差异，而是将可复用能力逐层沉淀：从逐端适配走向多端复用，从单一的页面复用扩展到布局、状态与交互等完整体验规则，并进一步形成能够覆盖更多设备形态的多形态能力。

2.1 从逐端适配走向多端复用

在单个平台上完成折叠屏适配并不困难：监听窗口变化、增加布局断点，再针对大屏调整页面结构即可。但当同一业务需要覆盖多个平台时，真正昂贵的是后续维护。

同一套分栏规则，需要在不同技术栈中分别实现；同一个选中状态，需要分别处理折叠与展开后的恢复；同一种导航行为，需要同时适配小屏的整页跳转和大屏的区域切换；同一套转场与返回策略，也需要逐端开发、测试和对齐。

随着平板、桌面窗口、AI 超级终端等更多形态加入，单端方案沉淀下来的往往是多套相似但难以共享的实现。平台越多、形态越多，体验一致性和长期演进的成本就越高。

Kuikly 将各端窗口变化统一传递到共享 Kotlin 层，业务只需定义一套响应式适配规则，即可复用于多个平台。平台侧主要承接必要的容器和系统能力，从而减少重复开发，也降低后续对齐与维护成本。

2.2 完整一致的体验规则

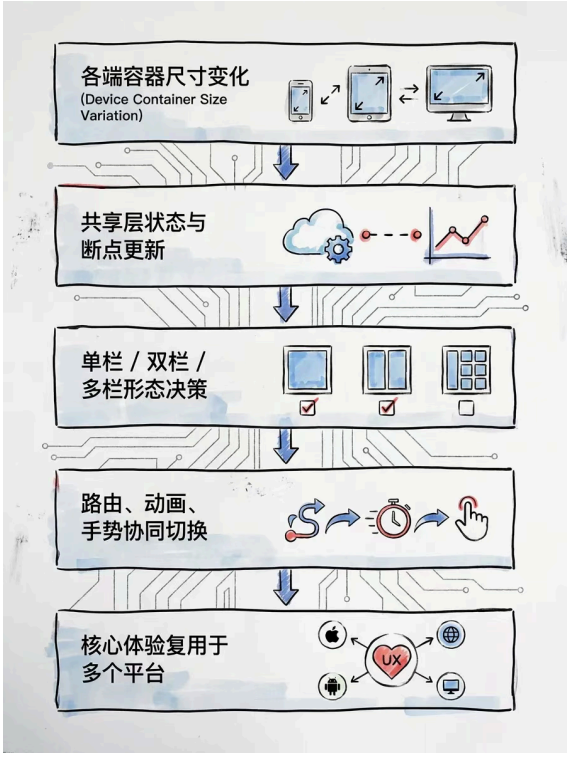
页面能否适应折叠屏，不只取决于如何绘制，还取决于尺寸、状态、布局与交互能否协同变化。折叠屏场景中的跨端价值，也不只是"一套代码画出相同页面"，更重要的是将折叠屏适配中容易重复的规则统一沉淀到共享层：

- **统一尺寸模型：**各端使用相同的容器尺寸和布局断点语义；
- **统一状态模型：**选中项、输入内容和页面层级不依赖具体平台容器；
- **统一布局策略：**单栏、双栏、多栏遵循同一套切换规则；
- **统一导航语义：**业务只表达进入、返回和选择，由不同形态决定整页跳转还是区域切换；
- **统一交互原则：**动画节奏、返回优先级和手势作用域可以在共享层协同设计。

Kuikly 复用的不只是 UI 代码，还包括一套完整、一致的体验规则。业务完成一次适配，核心逻辑即可复用于多个平台；同时，各端仍可保留符合自身系统特征的原生体验。

2.3 从"页面复用"走向"多形态能力复用"

一套可持续的跨端大小屏方案，需要形成完整链路：先感知窗口变化，再由共享状态做出页面形态决策，最终让布局、路由和交互共同响应。



Kuikly 多形态能力复用路线

针对折叠与展开场景建立的布局断点、策略和状态管理能力，同样可以用于平板、分屏、小窗及后续大屏场景。当新的设备形态出现时，业务只需在既有能力上调整布局断点和策略，无需重新建设一套适配体系。

在通过 Kuikly 兼容折叠屏的同时，也沉淀为可共享的多形态能力：核心体验在共享 Kotlin 层持续复用，平台侧保留必要的原生增强空间。

三、Kuikly 的适配实践

Kuikly 在设计之初，就将窗口尺寸变化和多形态适配纳入框架能力，而不是将折叠屏作为一个新增平台单独适配。框架会把原生容器的尺寸变化传递到共享层，驱动页面响应式刷新，使同一套页面能够自然应对折叠、展开、分屏和小窗等不同形态。

在此基础上，Kuikly 还进一步解决页面结构变化带来的路由、动画和手势问题，让多形态切换不仅能够正确显示，也能保持完整、连续的交互体验。

3.1 路由：统一整页跳转与区域切换

小屏页面通常借助 Android Activity、iOS ViewController 等系统载体完成整页跳转；进入大屏后，列表与详情往往同时存在，原来的"打开新页面"会变成指定区域内的内容切换，无法继续完全依赖系统路由。

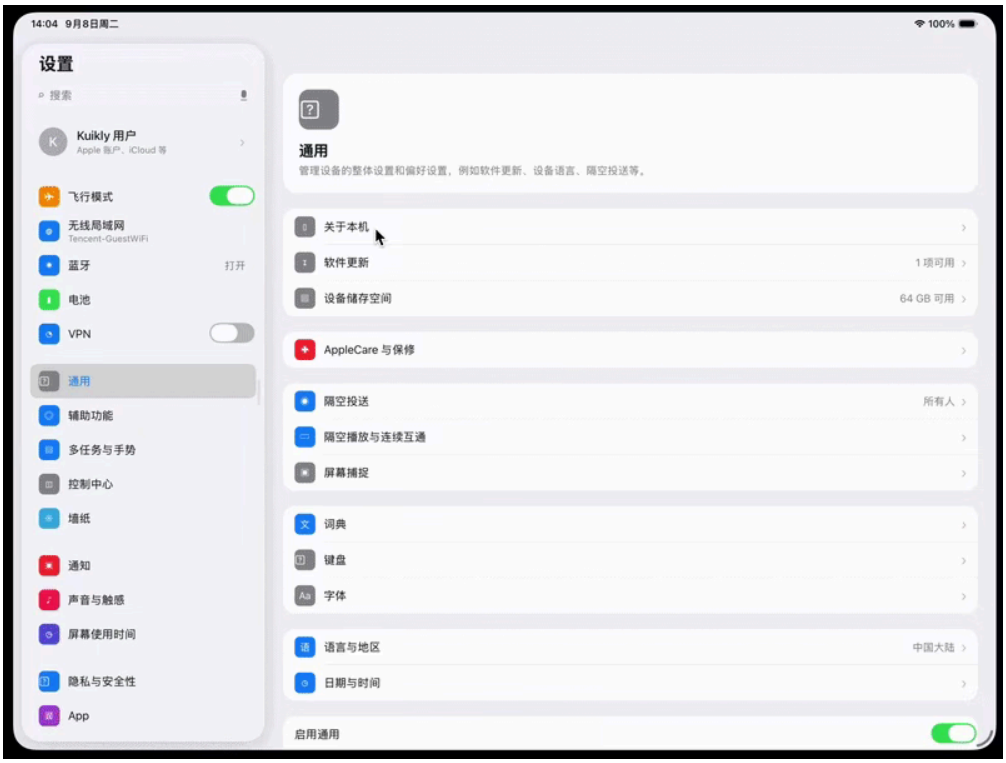
这会带来三个问题：

- 小屏与大屏采用不同的页面承载形式，但应保持一致的导航语义；

- 折叠与展开过程中，路由栈需要迁移，并恢复当前选中项与页面状态；
- 区域之间既要支持参数传递，也要避免与具体页面过度绑定。

Kuikly 的适配思路是在自适应页面或组件内部建立统一的导航状态：业务仍然表达"进入详情""返回上一级"等操作，组件根据当前窗口形态，将其映射为小屏的页面跳转或大屏的区域切换，路由状态由响应式数据驱动，并统一管理内容层级、生命周期和状态恢复。





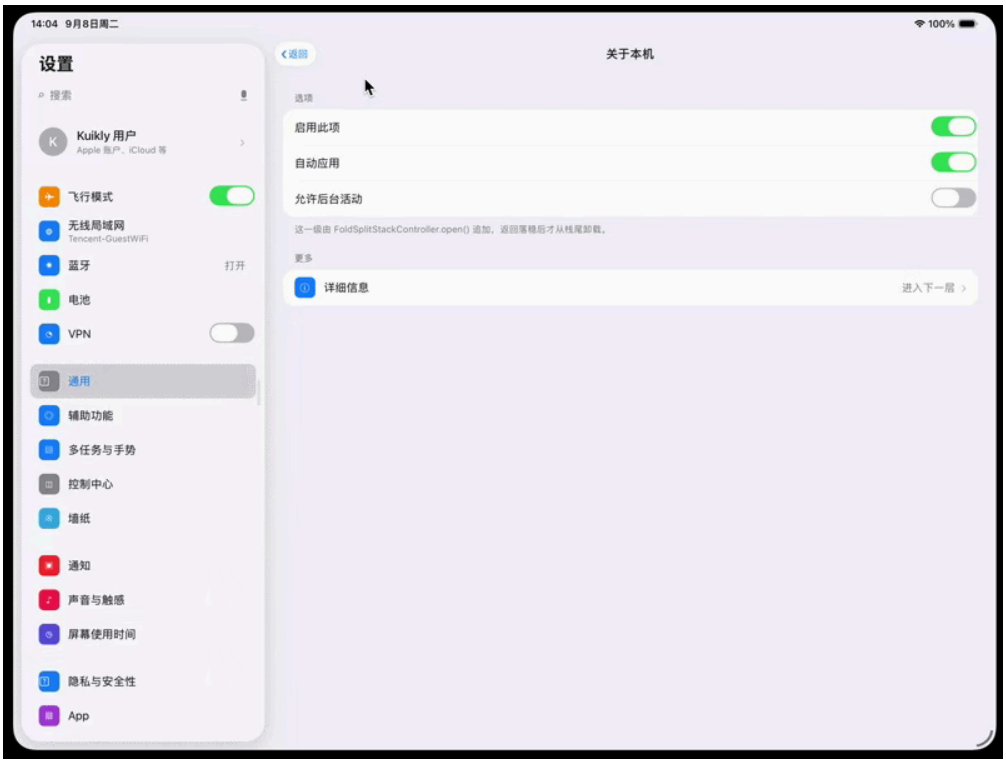
基于 Kuikly 实现的自适应屏幕设置页「页内跳转与区域切换」

3.2 动画：从系统转场扩展到局部转场

小屏中的页面切换通常可以复用系统转场动画；大屏中的内容切换则多发生在页面内部的局部区域，系统转场不再适用。如果没有统一处理，同一个导航操作会在两种形态下呈现完全不同的节奏，甚至出现内容闪烁或瞬间跳变。

因此，Kuikly 将动画与自适应导航状态结合：小屏保持符合平台习惯的进退场关系，大屏则在目标区域内执行对应的内容转场；从单栏切换到双栏时，还可以同步编排列表宽度、详情位置和导航元素，使内容关系在窗口变化中保持连续。





基于 Kuikly 实现的自适应屏幕设置页「页内返回动画」

3.3 手势：统一的返回策略及事件处理

在小屏中，各页面通常由独立系统容器承载，系统返回手势可以直接作用于当前页面；在大屏中，右侧区域可能存在多层内容，此时一次返回究竟应该关闭整个页面，还是只返回右侧区域的上一级，需要由自适应导航统一决策。

Kuikly 的处理重点包括：

- 小屏二级页面响应系统返回，并允许业务自定义返回策略；
- 大屏优先回退当前区域的内部层级，区域栈清空后再交由外层页面处理；
- 不同层级的内容区域主动消费对应手势，避免点击、拖动和滚动穿透到底层页面。

通过将返回栈与手势作用域关联起来，大小屏能够共享相同的导航语义，同时保持符合当前页面结构的交互行为。





基于 Kuikly 实现的自适应屏幕设置页「页内返回手势」

3.4 平台适应：统一开发体验，也保留原生质感

跨端开发需要统一各平台的使用方式，但"统一"并不意味着所有平台只能呈现相同效果。如果框架只提供各系统都具备的公共能力，业务虽然获得了代码复用，却可能错过新系统最具辨识度的体验。

Kuikly 依托原生渲染架构，将平台差异收敛在框架内部，让开发者不必在跨端复用与平台体验之间做取舍。同一套折叠屏页面可以共享布局、路由和交互逻辑，同时也能自然融入 iOS Liquid Glass 等平台特性，并在其他平台保持符合各自系统特征的视觉表现。

同样的差异也体现在状态栏上，iPhone Duo 的状态栏采用了非对称与侧边集成设计，其他厂商在挖孔、类灵动岛方案和系统栏高度上也各有不同，目前还没有统一标准。Kuikly 把各平台的状态栏、导航栏、刘海、挖孔、圆角等物理限制，统一抽象成一套安全距离，并随尺寸变化同步更新。开发者一次布局，即可在各端正确避让。

依托原生渲染架构，Kuikly 将持续关注操作系统、硬件形态和交互方式的演进，让业务能够以更低成本跟进新系统、新设备与新体验。

上述自适应屏幕设置页与多形态 App 适配的 Sample code，可查看：https://github.com/Tencent-TDS/KuiklyUI/tree/feat/kuikly_fold

示例 Sample 组件后续会整理成组件发布，方便大家直接使用；欢迎 Star 关注 Kuikly 仓库，获取最新更新。

四、总结与展望

设备形态的发展，为跨端开发提出了一个新的命题。

过去，跨端框架主要解决"同一个页面如何运行在多个系统上"；随着折叠屏、平板和桌面窗口不断发展，它还需要解决"同一个 App 如何持续适应变化的空间与输入方式"。

单端适配可以解决一个平台的当前问题，而跨端方案更关注如何将一次适配沉淀为可复用、可演进的基础能力。一个跨端框架的价值，也不再只取决于它能抹平多少平台差异，更取决于它能否帮助业务在控制工程成本的同时，及时释放新设备和新系统的体验价值。

Kuikly 的折叠屏适配实践验证了这一思路：同一份业务状态可以在小屏中以单页呈现，在大屏中以多栏展开；同一套页面逻辑可以响应窗口变化重新组织；核心代码则可以面向多个平台持续复用。

未来，随着设备形态和交互方式继续演进，跨端可能不再只是提升研发效率的工程选择，也将成为 App 快速跟进硬件创新的重要基础设施。Kuikly 将继续完善折叠屏、大屏和多窗口场景能力，帮助开发者在不断变化的设备生态中创造更高效、更自然的 App 体验。

折叠的是屏幕，不打折的是体验。

五、关于 Kuikly

当前Kuikly已经开源，有兴趣和有需要的产品，可以通过以下方式访问 Kuikly 仓库和文档，欢迎Star、Watch与体验：

👉 Github 仓库 | 📚 官方文档

● 文中Sample code链接：GitHub - Tencent-TDS/KuiklyUI at feat/kuikly_fold · GitHub

Kuikly框架属于腾讯端服务联盟（tds.qq.com）的重要成员，欢迎关注及了解更多信息：

● 腾讯端服务官网：TDS 腾讯端服务

● Kuikly官网：跨平台框架

腾讯端服务服务号	Kuikly 公众号	在线咨询





阅读 1.7万

留言

写留言

还没有留言，来写首评